

Effect of Bin Size On ϵ -DP Perturbed Histogram Synthesizer For Continuous Variables

Data Set and Explanation

For this project I've chose to choose a healthcare data set from Kaggle. The data set can be found at: https://www.kaggle.com/data-sets/prasad22/healthcare-data-set?select=healthcare_data_set.csv. Below will be a summary of the data set and its variables taken from Kaggle.

Name: This column represents the name of the patient associated with the healthcare record.

Age: The age of the patient at the time of admission, expressed in years. Gender: Indicates the gender of the patient, either "Male" or "Female."

Blood Type: The patient's blood type, which can be one of the common blood types (e.g., "A+", "O-", etc.).

Medical Condition: This column specifies the primary medical condition or diagnosis associated with the patient, such as "Diabetes," "Hypertension," "Asthma," and more.

Date of Admission: The date on which the patient was admitted to the healthcare facility.

Doctor: The name of the doctor responsible for the patient's care during their admission.

Hospital: Identifies the healthcare facility or hospital where the patient was admitted.

Insurance Provider: This column indicates the patient's insurance provider, which can be one of several options, including "Aetna," "Blue Cross," "Cigna," "UnitedHealthcare," and "Medicare."

Billing Amount: The amount of money billed for the patient's healthcare services during their admission. This is expressed as a floating-point number.

Room Number: The room number where the patient was accommodated during their admission.

Admission Type: Specifies the type of admission, which can be "Emergency," "Elective," or "Urgent," reflecting the circumstances of the admission.

Discharge Date: The date on which the patient was discharged from the healthcare facility, based on the admission date and a random number of days within a realistic range.

Medication: Identifies a medication prescribed or administered to the patient during their admission. Examples include "Aspirin," "Ibuprofen," "Penicillin," "Paracetamol," and "Lipitor."

Test Results: Describes the results of a medical test conducted during the patient's admission. Possible values include "Normal," "Abnormal," or "Inconclusive," indicating the outcome of the test.

Methadology/Introduction

Through this project I aim to see the affect bin size has on the ϵ -DP perturbed histogram synthesizer for continuous variables. From this data set I will be synthesizing both Age and Billable Amount. While synthesizing I will keep both the epsilon value/privacy budget and added noise constant while comparing the effects of various bin sizes. Our data set has an n-value of 55,500 and based from this we will look at four different m values: $\ln(n)$, 100, 200, and $\sqrt{n}$ (10, 100, 200, 235). These two variables were chosen because they are continuous variables and also have drastically different range of values.

Data Set Exploration and Cleaning

```
#Load In The Data Set
hcdata <- read.csv("C:/Users/ayron/Downloads/healthcare_dataset.csv")
summary(hcdata)
```

```
##      Name                Age                Gender                Blood.Type
## Length:55500           Min.   :13.00       Length:55500           Length:55500
## Class :character       1st Qu.:35.00       Class :character       Class :character
## Mode  :character       Median :52.00       Mode  :character       Mode  :character
##                               Mean   :51.54
##                               3rd Qu.:68.00
##                               Max.   :89.00
## Medical.Condition Date.of.Admission      Doctor                Hospital
## Length:55500      Length:55500           Length:55500           Length:55500
## Class :character   Class :character       Class :character       Class :character
## Mode  :character   Mode  :character       Mode  :character       Mode  :character
##
##
##
## Insurance.Provider Billing.Amount      Room.Number      Admission.Type
## Length:55500       Min.   :-2008       Min.   :101.0       Length:55500
## Class :character   1st Qu.:13241     1st Qu.:202.0       Class :character
## Mode  :character   Median :25538     Median :302.0       Mode  :character
##                               Mean   :25539     Mean   :301.1
##                               3rd Qu.:37821     3rd Qu.:401.0
##                               Max.   :52764     Max.   :500.0
## Discharge.Date      Medication          Test.Results
## Length:55500        Length:55500        Length:55500
## Class :character     Class :character     Class :character
## Mode  :character     Mode  :character     Mode  :character
##
##
##
```

```
#Checking For Missing Values
any(is.na(hcdata[, c("Age", "Billing.Amount"))))
```

```
## [1] FALSE
```

We see that there are no missing values so we can continue as usual

Creating A Function For Synthesis

```
synthesize_dp <- function(data, column, m, a_min, a_max, eps = 1, seed = 730) {
  y_vector <- data %>%
    pull({{ column }})
  # Calculate the sample size
```

```

n <- data %>%
nrow()
# Create bounded confidential vector
y_vector_bounded <- (y_vector - a_min) / (a_max - a_min)
# Calculate the length of each bin
h <- 1 / m
# Create the bins
bins <- seq(0, 1, h)
# Calculate C_j's
C <- rep(0, m)
for (j in 1:m) {
  C[j] <- sum(
    y_vector_bounded >= bins[j] &
    (y_vector_bounded < bins[j + 1] | (j == m & y_vector_bounded == 1))
  )
}
# Set seed to obtain reproducible results
base::set.seed(seed)
# Sample vector
eta <- rmutil::rlaplace(n = m,
m = 0,
s = 1/eps)
# Create D vector
D <- C + eta
# Create D_tilde vector
D_tilde <- rep(0, m)
for (j in 1:m){
D_tilde[j] <- max(D[j], 0)
}
# Calculate q_hat_pert vector
q_hat_pert <- D_tilde/sum(D_tilde)
# Simulate DP microdata value within bounds
y_vector_bounded_star <- rep(NA, n)
for (i in 1:n){
bin_i <- which(stats::rmultinom(1, 1, q_hat_pert) == 1)
range_lower <- bins[bin_i]
range_upper <- bins[bin_i + 1]
obs_i <- runif(1, range_lower, range_upper)
y_vector_bounded_star[i] <- obs_i
}
# Create DP synthetic vector in its original scale
y_vector_star <- y_vector_bounded_star*(a_max - a_min) + a_min
# Combine two vectors in a single data frame
df <- tibble(
type = rep(c("Confidential", "DPsynthetic"), each = n),
vectors = c(y_vector, y_vector_star)
) %>%
mutate(type = factor(type,
levels = c("Confidential", "DPsynthetic")))
df_long <- reshape2::melt(df)
# Plot density distributions of the confidential and synthetic vectors
p <- ggplot2::ggplot(data = df_long, aes(value, colour = type,
linetype = type)) +

```

```

geom_density() +
scale_colour_manual(values = c("#E69F00", "#999999"),
                    guide = guide_legend(override.aes = list(
                      linetype = c(1, 2)))) +
scale_linetype_manual(values = c(1, 2), guide = "none") +
xlab(as.character(substitute(column))) +
theme_bw(base_size = 15, base_family = "") +
theme(legend.title = element_blank())

print(p)

#Store list of values for other uses
invisible(list(
  m = m,
  eps = eps,
  a_min = a_min,
  a_max = a_max,
  y_star = y_vector_star,
  plot = p
))
}

```

Create Function To Calculate Propensity Score Mean-Squared Error

```

calc_pMSE <- function(synthetic_vector, column) {

  n <- nrow(hcdata)

  # Create final synthetic data set
  synthetic_final <- hcdata %>%
    mutate({{ column }} := synthetic_vector)

  # Merge confidential and synthetic
  merged_data <- dplyr::bind_rows(
    hcdata,
    synthetic_final
  )

  # Add S indicator
  merged_data <- merged_data %>%
    mutate(S = c(rep(0, n), rep(1, n)))

  # Logistic regression with all relevant variables
  log_reg <- stats::glm(formula = S ~ (Age +
    Billing.Amount +
    Room.Number +
    as.factor(Gender) +
    as.factor(Blood.Type) +
    as.factor(Medical.Condition) +
    as.factor(Insurance.Provider) +
    as.factor(Admission.Type) +

```

```

                                as.factor(Medication) +
                                as.factor(Test.Results))^2,
                                family = "binomial",
                                data = merged_data)

# Predicted probabilities
pred <- stats::predict(log_reg, data = merged_data)
probs <- exp(pred) / (1 + exp(pred))

# Calculate pMSE
pMSE <- 1/(2*n) * sum((probs - 1/2)^2)

return(pMSE)
}

```

Create Function For Calculating EMR,FMR,TMR

```

CalculateKeyQuantities_cont <- function(condata, syndata,
                                       unsyn.vars, syn.vars,
                                       n, r){

  condata <- condata
  syndata <- syndata
  n <- n
  c_vector <- rep(NA, n)
  T_vector <- rep(NA, n)
  for (i in 1:n){
    radius <- r*eval(parse(text=paste("condata$", syn.vars, "[i]")))
    match <- (eval(parse(text=paste("syndata$", syn.vars,
                                   "<=condata$", syn.vars,
                                   "[i]+", radius, sep="",
                                   collapse="&")))) &
              eval(parse(text=paste("syndata$", syn.vars,
                                   ">=condata$", syn.vars,
                                   "[i]-", radius, sep="",
                                   collapse="&")))) &
              eval(parse(text=paste("condata$", unsyn.vars,
                                   "[i]==syndata$",
                                   unsyn.vars, sep="",
                                   collapse="&"))))

    match.prob <- ifelse(match, 1/sum(match), 0)
    if (max(match.prob) > 0){
      c_vector[i] <- length(match.prob[match.prob == max(match.prob)])
    }
    else
      c_vector[i] <- 0
    T_vector[i] <- is.element(i, rownames(condata)[match.prob == max(match.prob)])
  }
  K_vector <- (c_vector * T_vector == 1)
  F_vector <- (c_vector * (1 - T_vector) == 1)
  s <- length(c_vector[c_vector == 1 & is.na(c_vector) == FALSE])
  res_r <- list(c_vector = c_vector,

```

```

        T_vector = T_vector,
        K_vector = K_vector,
        F_vector = F_vector,
        s = s)
  return(res_r)
}

IdentificationRiskCal <- function(c_vector, T_vector,
                                K_vector, F_vector,
                                s, N){
  nonzero_c_index <- which(c_vector > 0)
  EMR <- sum(1/c_vector[nonzero_c_index] * T_vector[nonzero_c_index])
  TMR <- sum(na.omit(K_vector))/N
  FMR <- sum(na.omit(F_vector))/s
  res_r <- list(EMR = EMR,
               TMR = TMR,
               FMR = FMR)
  return(res_r)
}

```

Run Synthesis Using Different m Values On Billable Amount and Plot Them

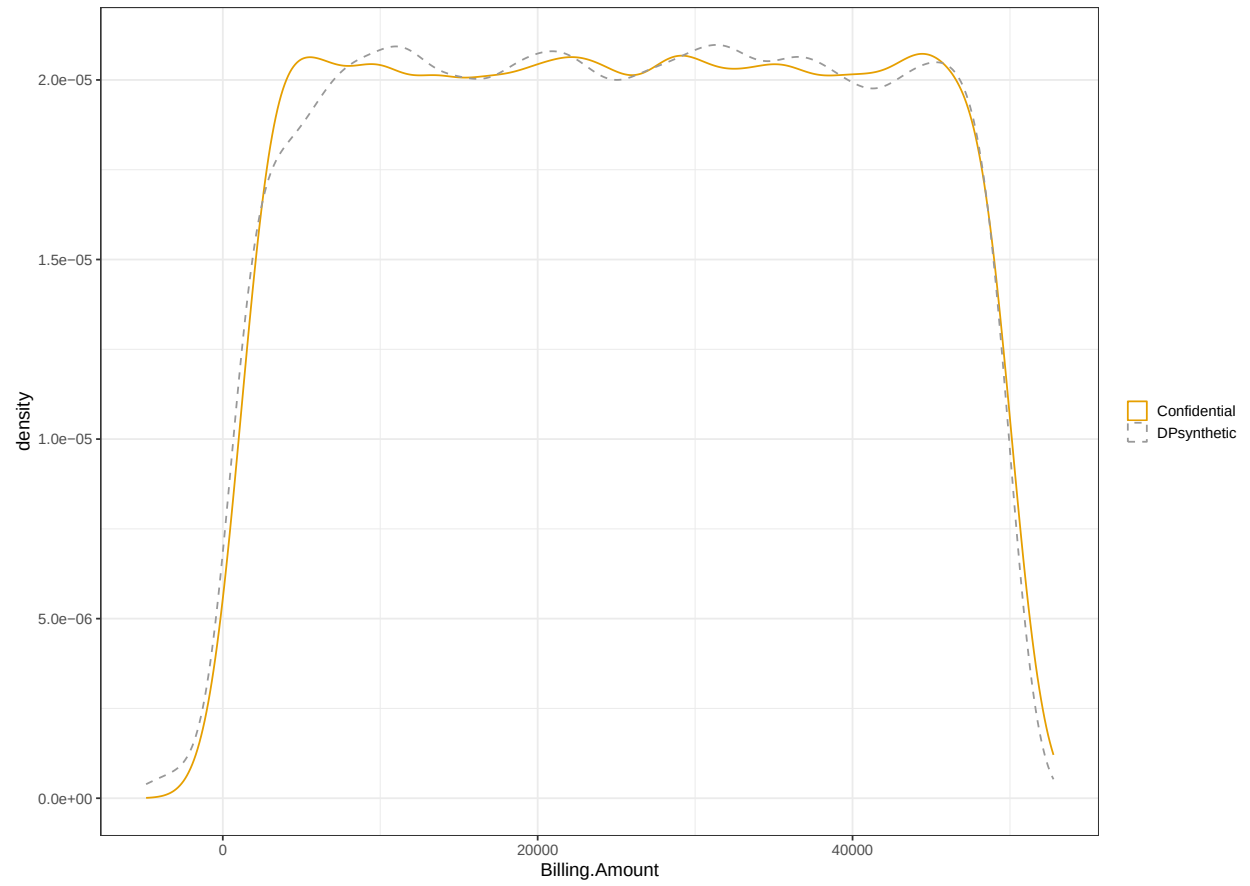
```

#Set N and m_values
n <- nrow(hcdata)
m_values <- c(floor(log(n)), 100, 200, floor(sqrt(n)))

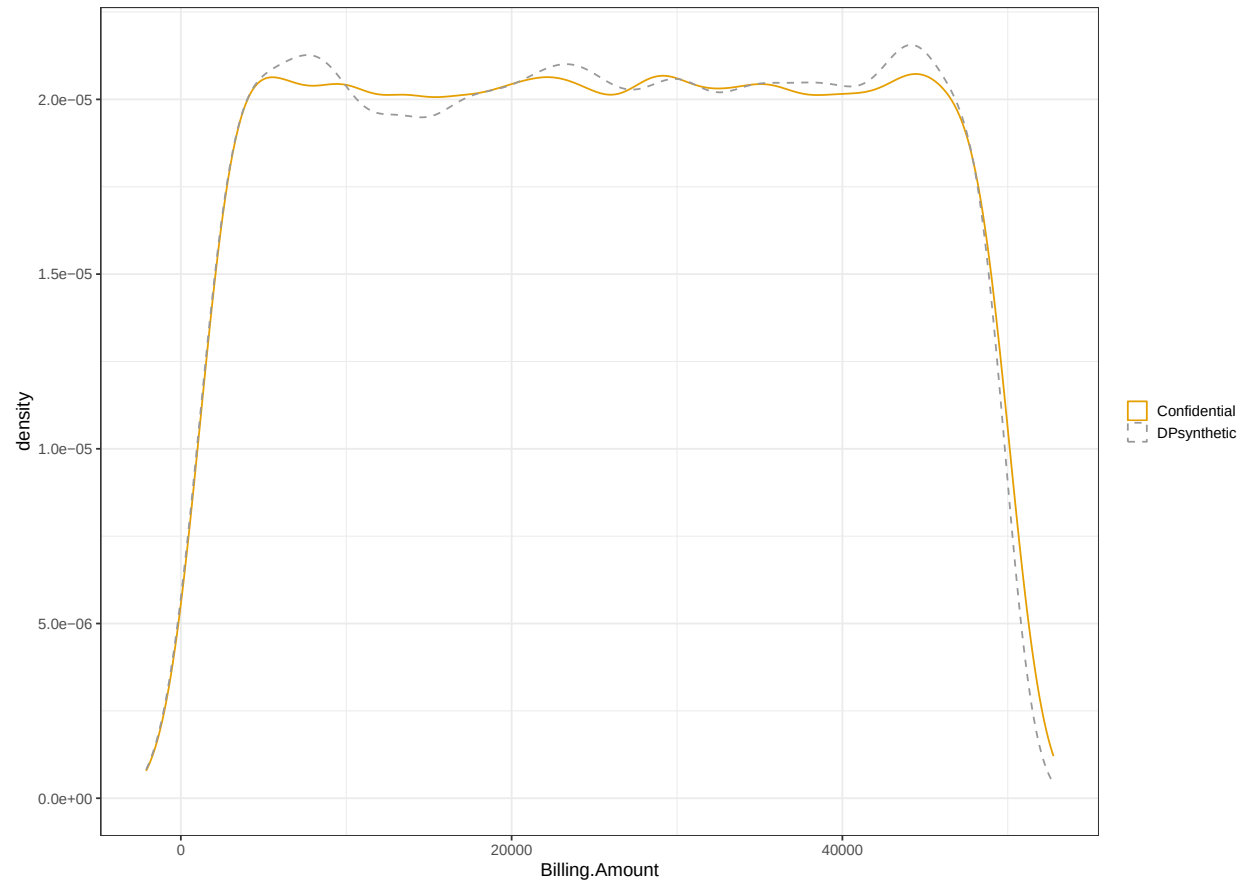
#Synthesize by applying the list of m values with range [-10,000 -50,000]
results_billing <- lapply(m_values, function(m) {
  synthesize_dp(hcdata, Billing.Amount, m = m, a_min = -5000, a_max = 50000)
})

## Using type as id variables

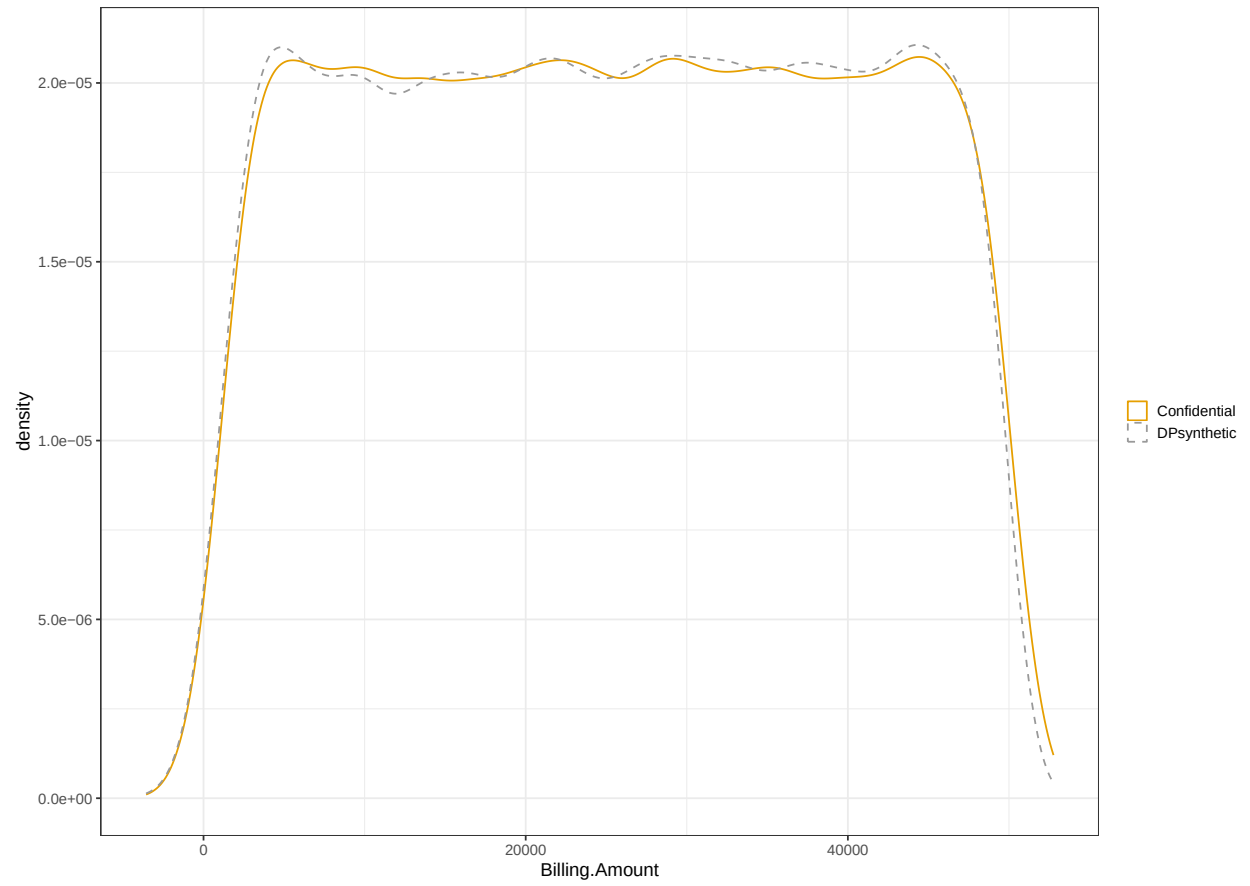
```



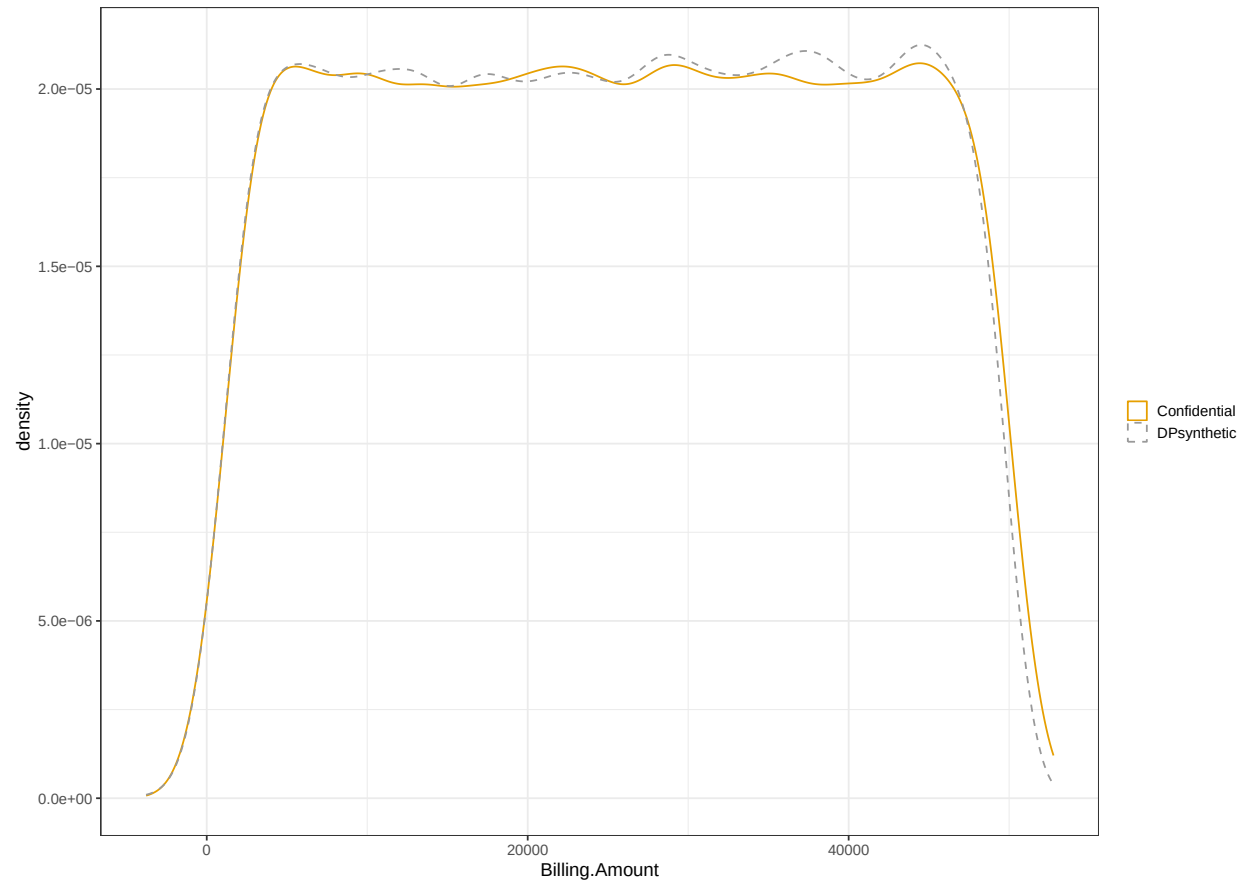
```
## Using type as id variables
```



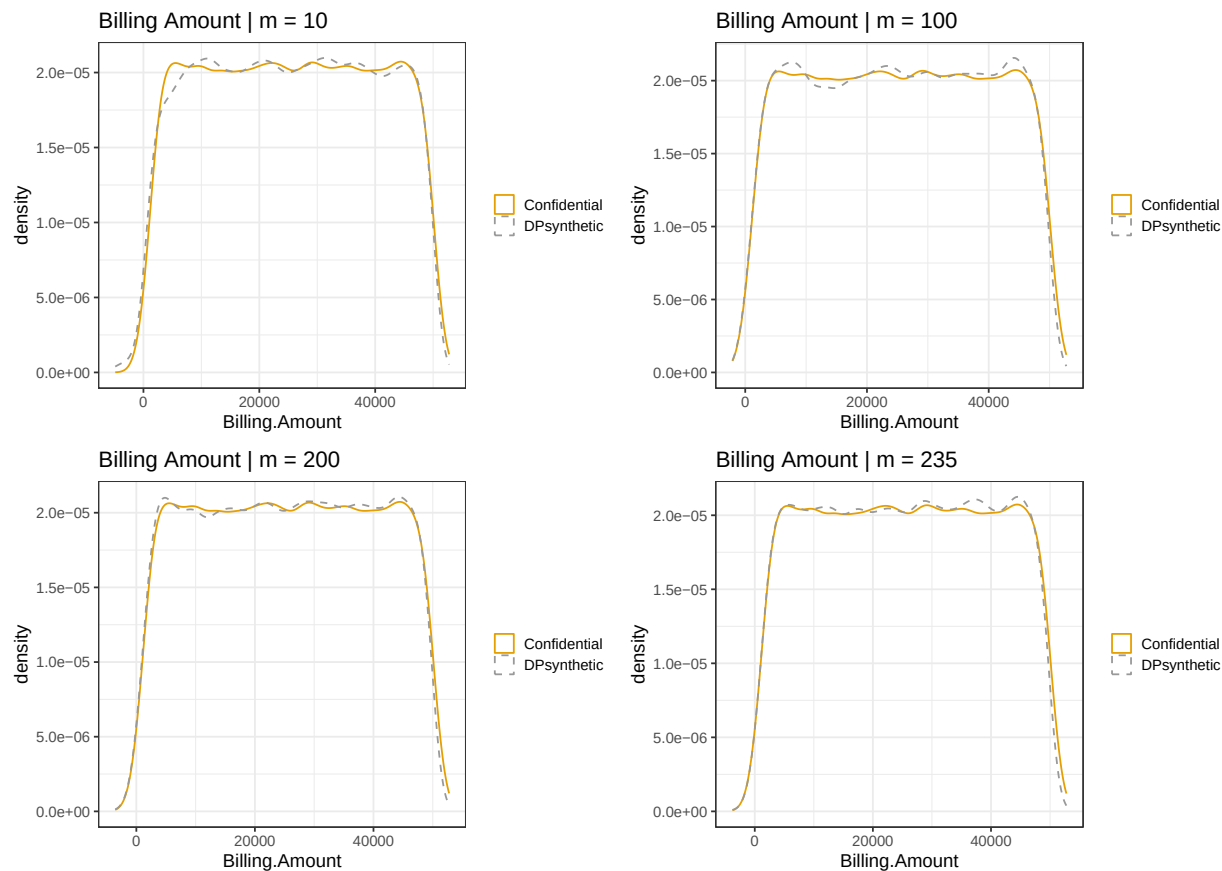
```
## Using type as id variables
```

```
## Using type as id variables
```



```
plots_billing <- lapply(seq_along(m_values), function(i) {  
  results_billing[[i]]$plot + ggtitle(paste("Billing Amount | m =", m_values[i]))  
})  
  
wrap_plots(plots_billing, ncol = 2)
```

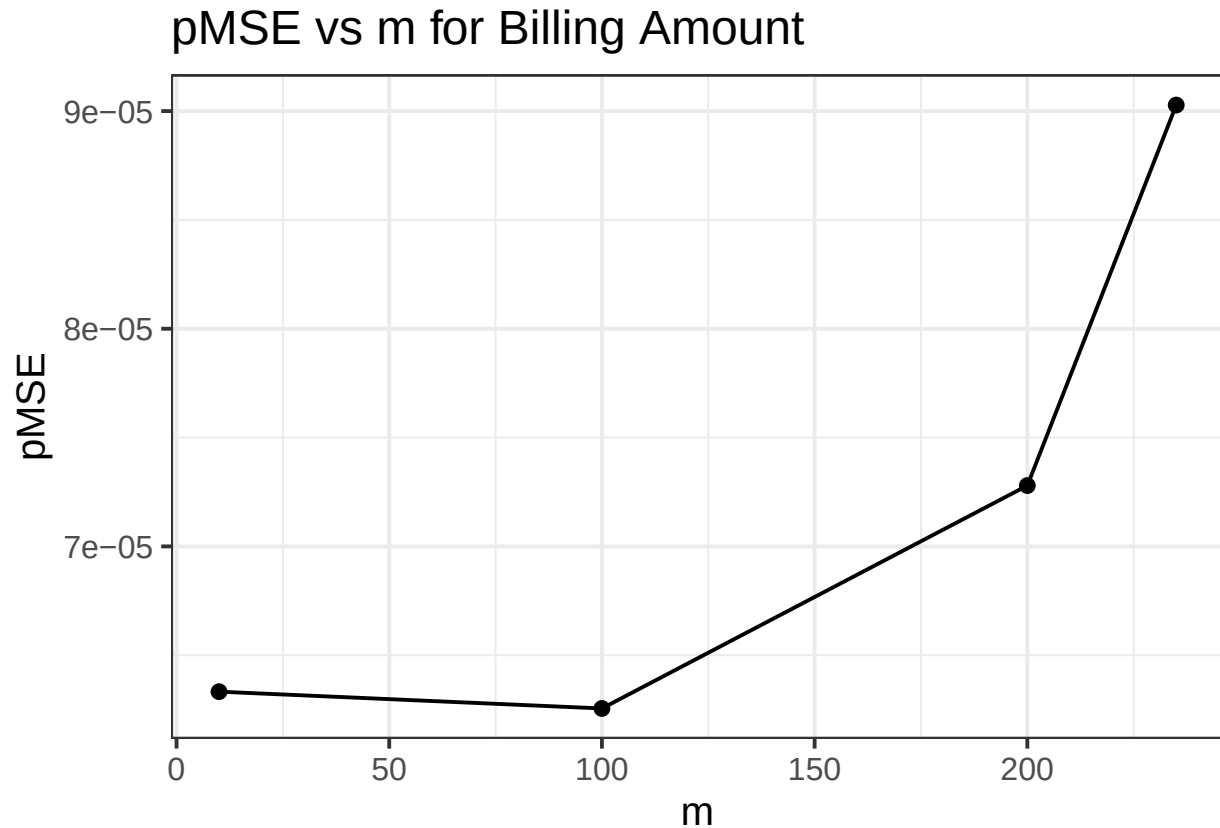


Get pMSE Values And Plot Them Across M Values

```
# Get pMSE values across m values for Billing Amount
pMSE_values_billing <- sapply(seq_along(m_values), function(i) {
  calc_pMSE(results_billing[[i]]$y_star, Billing.Amount)
})

# Build data frame
pMSE_df_billing <- tibble(
  m = m_values,
  pMSE = pMSE_values_billing
)

# Plot
ggplot(pMSE_df_billing, aes(x = m, y = pMSE)) +
  geom_line() +
  geom_point() +
  xlab("m") +
  ylab("pMSE") +
  ggtitle("pMSE vs m for Billing Amount") +
  theme_bw(base_size = 15)
```



Calculate and plot EMR,FMR,TMR for Billable Amount

```

risk_values <- sapply(seq_along(m_values), function(i) {
  syn_data <- hcdata %>% mutate(Billing.Amount = results_billing[[i]]$y_star)

  kq <- CalculateKeyQuantities_cont(condata = hcdata,
    syndata = syn_data,
    unsyn.vars = c("Gender",
                   "Blood.Type",
                   "Medical.Condition",
                   "Insurance.Provider",
                   "Admission.Type",
                   "Medication",
                   "Test.Results"),
    syn.vars = c("Billing.Amount"),
    n = nrow(hcdata),
    r = 0.2)

  risk <- IdentificationRiskCal(c_vector = kq[["c_vector"]],
    T_vector = kq[["T_vector"]],
    K_vector = kq[["K_vector"]],
    F_vector = kq[["F_vector"]],
    s = kq[["s"]],
    N = nrow(hcdata))

```

```

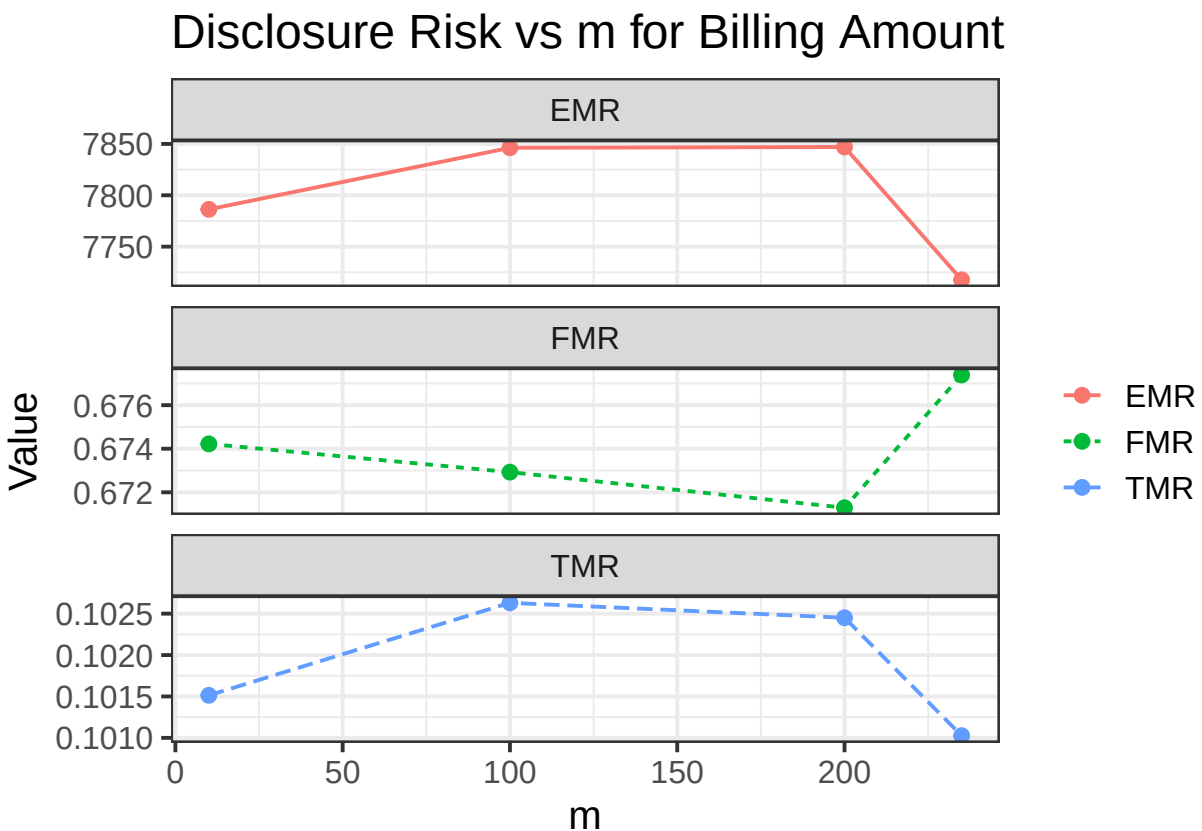
  c(EMR = risk[["EMR"]], TMR = risk[["TMR"]], FMR = risk[["FMR"]])
})

# Build data frame
risk_df <- tibble(
  m = m_values,
  EMR = risk_values["EMR", ],
  TMR = risk_values["TMR", ],
  FMR = risk_values["FMR", ]
)

risk_df_long <- risk_df %>%
  pivot_longer(cols = c(EMR, TMR, FMR),
    names_to = "measure",
    values_to = "value")

ggplot(risk_df_long, aes(x = m, y = value, colour = measure, linetype = measure)) +
  geom_line() +
  geom_point() +
  facet_wrap(~ measure, scales = "free_y", ncol = 1) +
  xlab("m") +
  ylab("Value") +
  ggtitle("Disclosure Risk vs m for Billing Amount") +
  theme_bw(base_size = 15) +
  theme(legend.title = element_blank())

```



Analysis For Billable Amount

Looking at our initial density plots from the synthesis we can start to see the effect the m value has on the distribution of the synthesized variable. Although the results look very similar, we can see that when $m=10$ the fit around both tails is considerably worse compared to the other m values. However, all m values capture the middle flat part of the distribution well. From this it is hard to differentiate the utility and fit of the synthesized variable visually, so we need a quantitative measure. In this case we use the Propensity Score Mean-Squared Error (pMSE), which measures the utility of synthetic data by calculating the average squared difference between the predicted propensity of a record being synthetic and the expected probability. A lower pMSE signifies higher utility, as there is little distinguishable difference between the original and synthesized variable. Looking at our pMSE plot we can see that $m=200$ and $m=235$ have considerably higher pMSE values. This can be attributed to the added Laplace noise — with more bins the relative effect of the noise becomes greater as each bin contains fewer observations. Between $m=10$ and $m=100$ the pMSE values are similar, but from the density plots $m=100$ has a noticeably better fit at the tails, making it the best choice for utility.

However, utility is not the only consideration when synthesizing data, as we also need to evaluate how well the synthesized data protects the confidential information. To measure this we look at three values: Expected Match Risk (EMR), the expected number of records an intruder correctly identifies across the whole data set; True Match Rate (TMR), the proportion of records where the intruder's best guess is actually correct; and False Match Rate (FMR), the proportion of the intruder's confident guesses that are actually wrong, where a higher value is better for privacy.

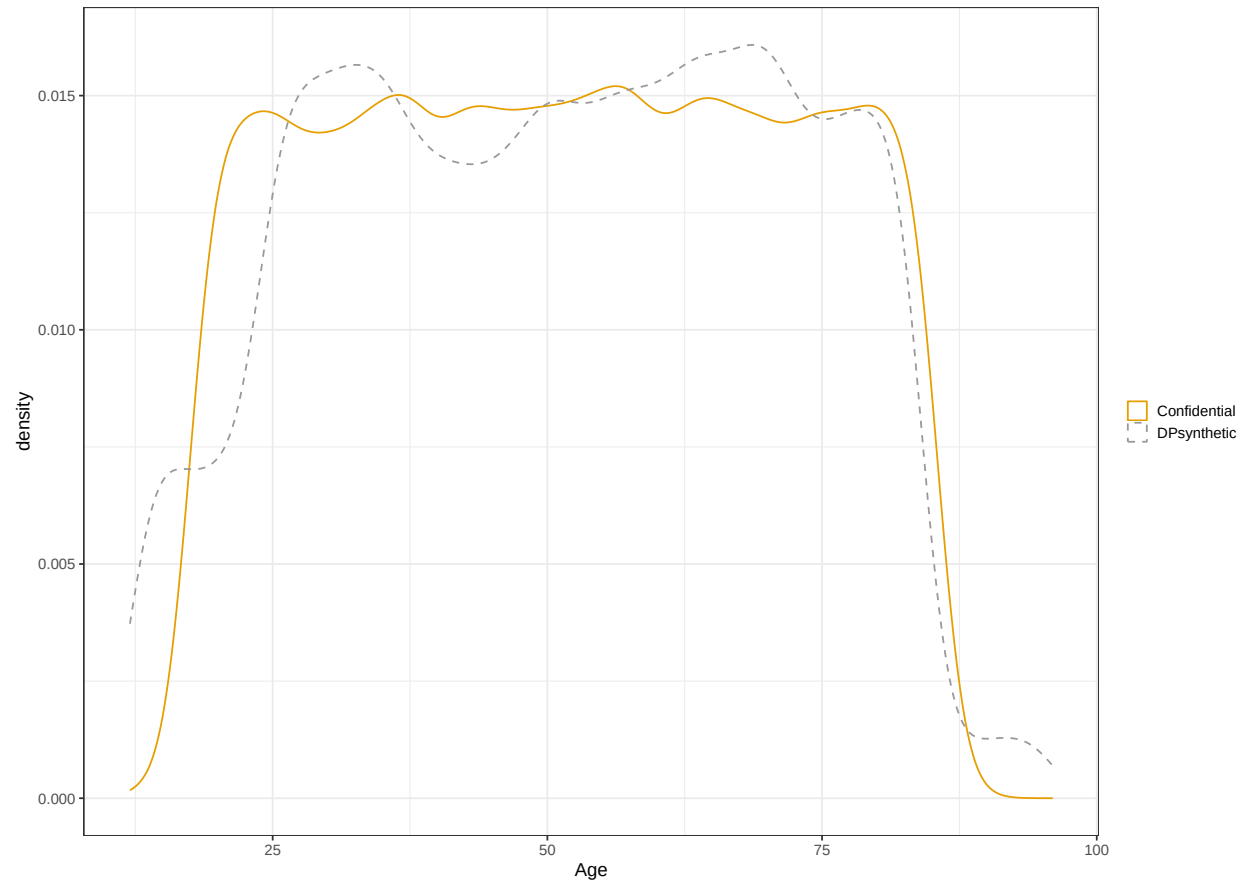
Looking at our results, $m=235$ performs best across all three measures. However it is important to note that the differences are very small — there is only about a 1.4% change in EMR across all m values, with similarly subtle differences in TMR and FMR. We can attribute $m=235$'s better privacy performance to the poorer fit of its synthetic distribution — with synthetic values that are more distorted due to noise, it becomes harder for an intruder to correctly match them back to the confidential records. This reaffirms a well known tradeoff in data synthesis: increasing utility comes at the cost of higher disclosure risk, and reducing risk often comes at the cost of utility. Given that the privacy differences across m values are minimal, $m=100$ represents the best overall choice as it maximizes utility without meaningfully sacrificing privacy protection.

Run Synthesis Using Different m Values On Age and Plot Them

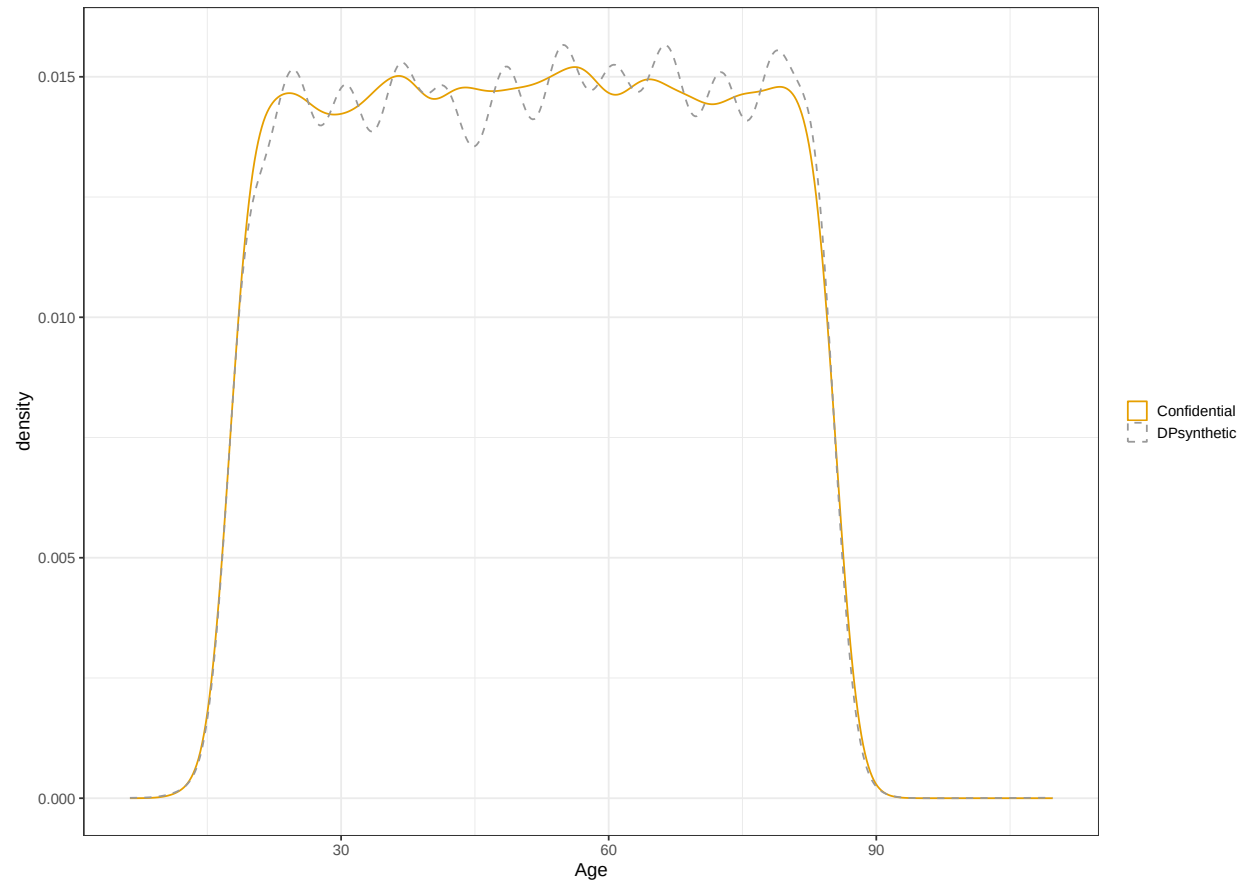
```
#Set N and m_values
n <- nrow(hcdata)
m_values <- c(floor(log(n)), 100, 200, floor(sqrt(n)))

#Synthesize by applying the list of m values with range [0-120]
results_Age <- lapply(m_values, function(m) {
  synthesize_dp(hcdata, Age, m = m, a_min = 0, a_max = 120)
})

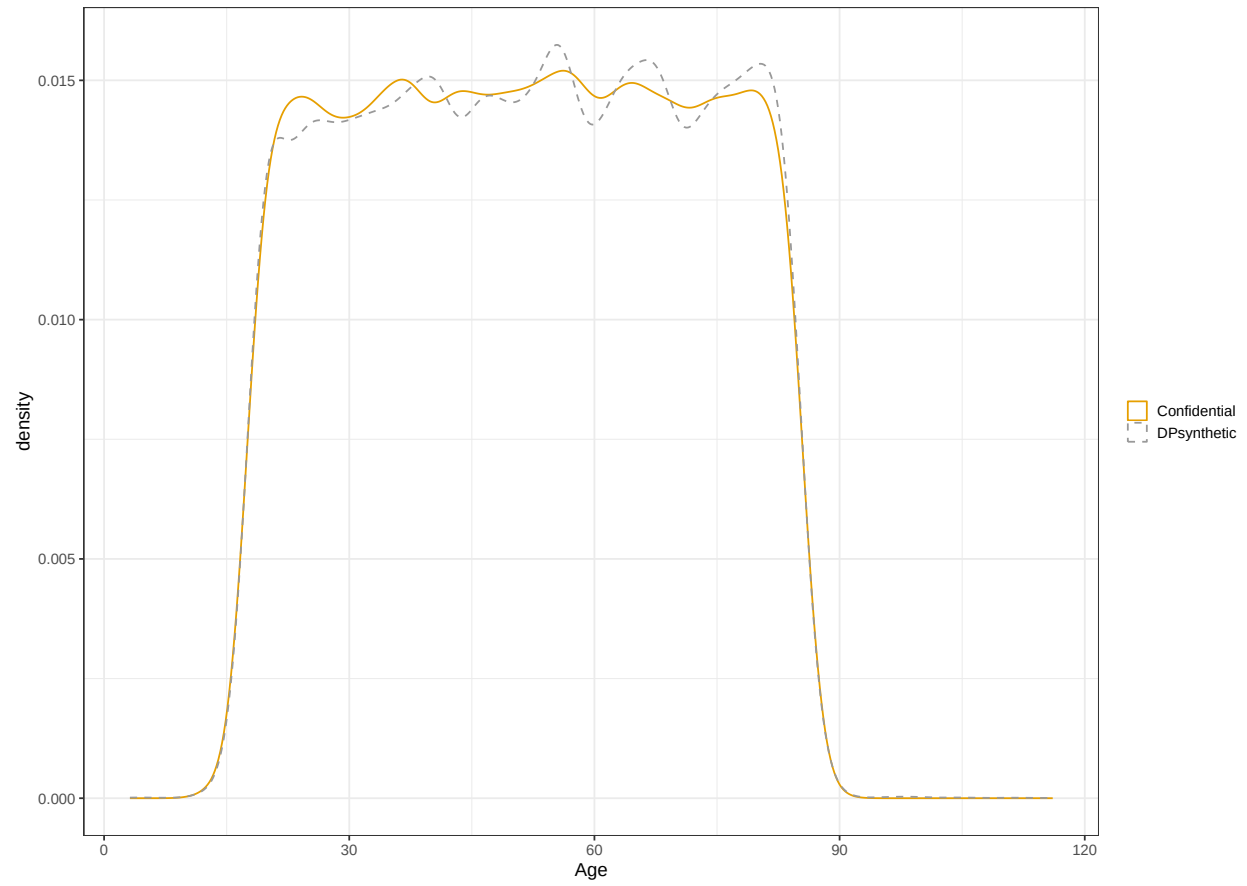
## Using type as id variables
```



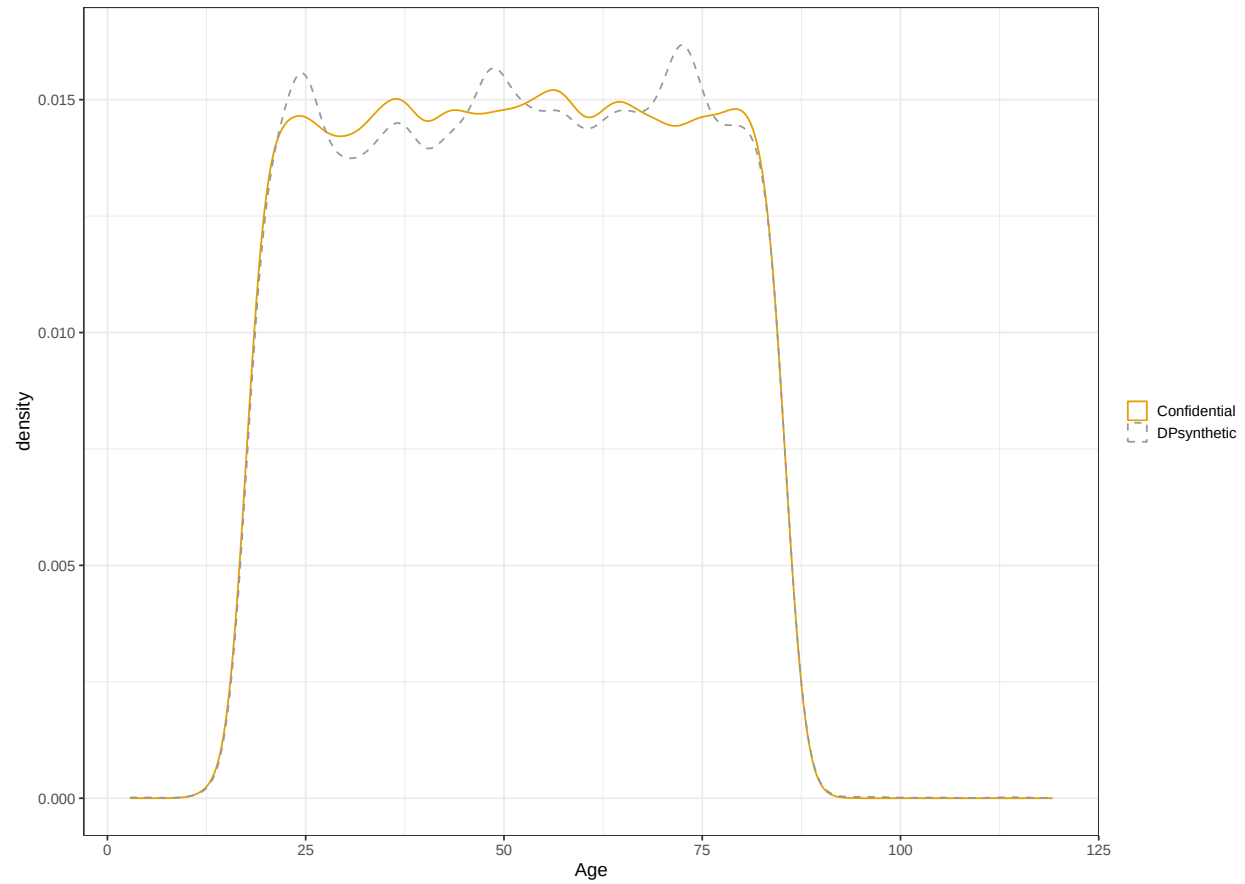
```
## Using type as id variables
```



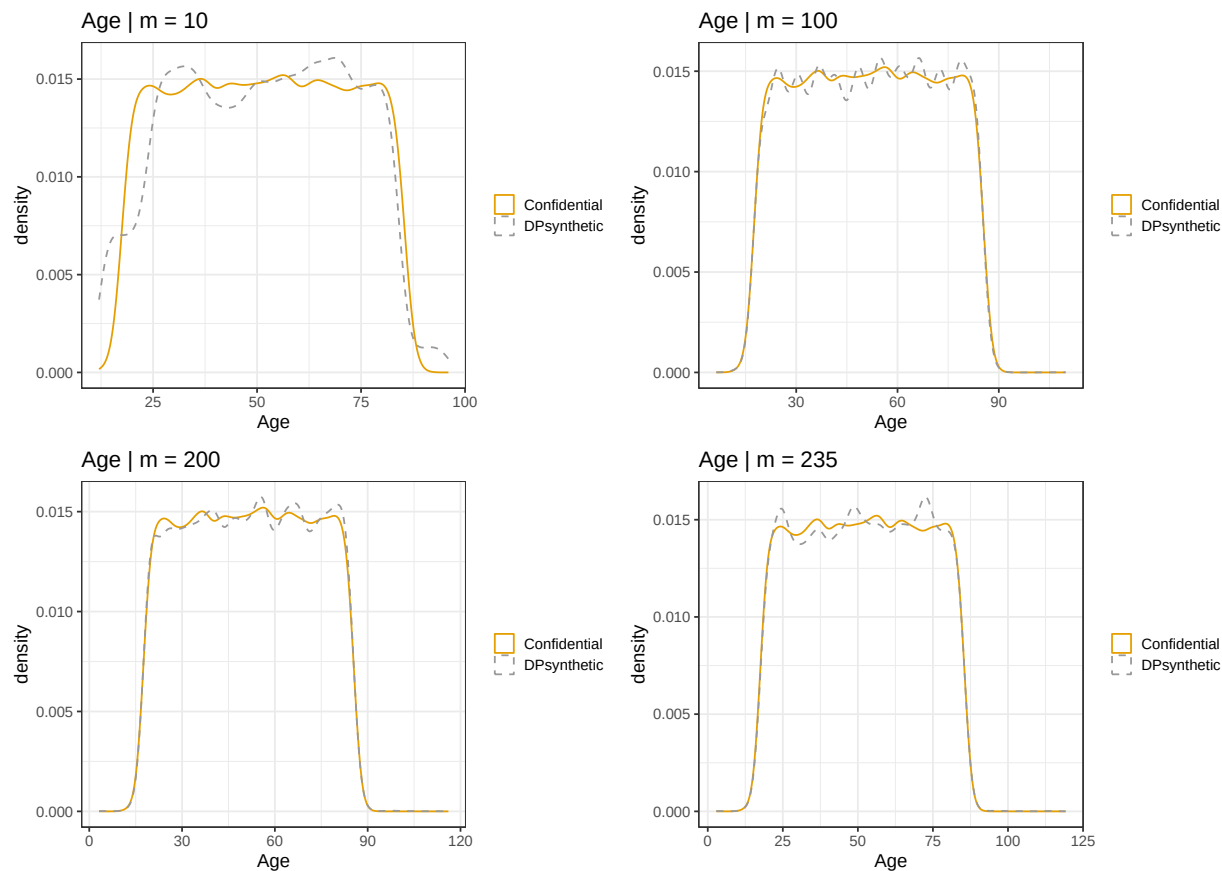
Using type as id variables



Using type as id variables



```
plots_Age <- lapply(seq_along(m_values), function(i) {  
  results_Age[[i]]$plot + ggtitle(paste("Age | m =", m_values[i]))  
})  
  
wrap_plots(plots_Age, ncol = 2)
```

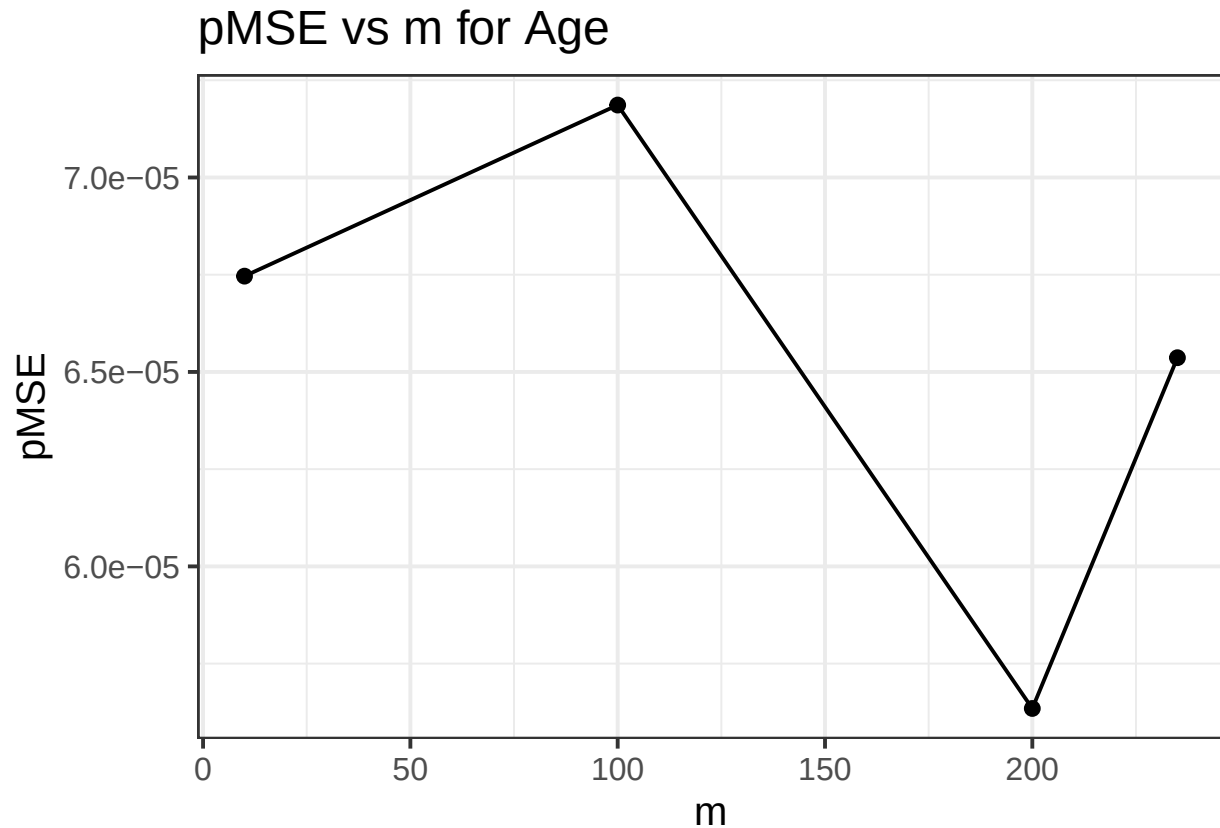


Get pMSE Values And Plot Them Across M Values

```
# Get pMSE values across m values for Age
pMSE_values_Age <- sapply(seq_along(m_values), function(i) {
  calc_pMSE(results_Age[[i]]$y_star, Age)
})

# Build data frame
pMSE_df_Age <- tibble(
  m = m_values,
  pMSE = pMSE_values_Age
)

# Plot
ggplot(pMSE_df_Age, aes(x = m, y = pMSE)) +
  geom_line() +
  geom_point() +
  xlab("m") +
  ylab("pMSE") +
  ggtitle("pMSE vs m for Age") +
  theme_bw(base_size = 15)
```



Calculate and plot EMR,FMR,TMR for Billable Amount

```
risk_values <- sapply(seq_along(m_values), function(i) {
  syn_data <- hcdata %>% mutate(Billing.Amount = results_billing[[i]]$y_star)

  kq <- CalculateKeyQuantities_cont(conddata = hcdata,
    syndata = syn_data,
    unsyn.vars = c("Gender",
                  "Blood.Type",
                  "Medical.Condition",
                  "Insurance.Provider",
                  "Admission.Type",
                  "Medication",
                  "Test.Results"),
    syn.vars = c("Age"),
    n = nrow(hcdata),
    r = 0.02)

  risk <- IdentificationRiskCal(c_vector = kq[["c_vector"]],
    T_vector = kq[["T_vector"]],
    K_vector = kq[["K_vector"]],
    F_vector = kq[["F_vector"]],
    s = kq[["s"]],
    N = nrow(hcdata))

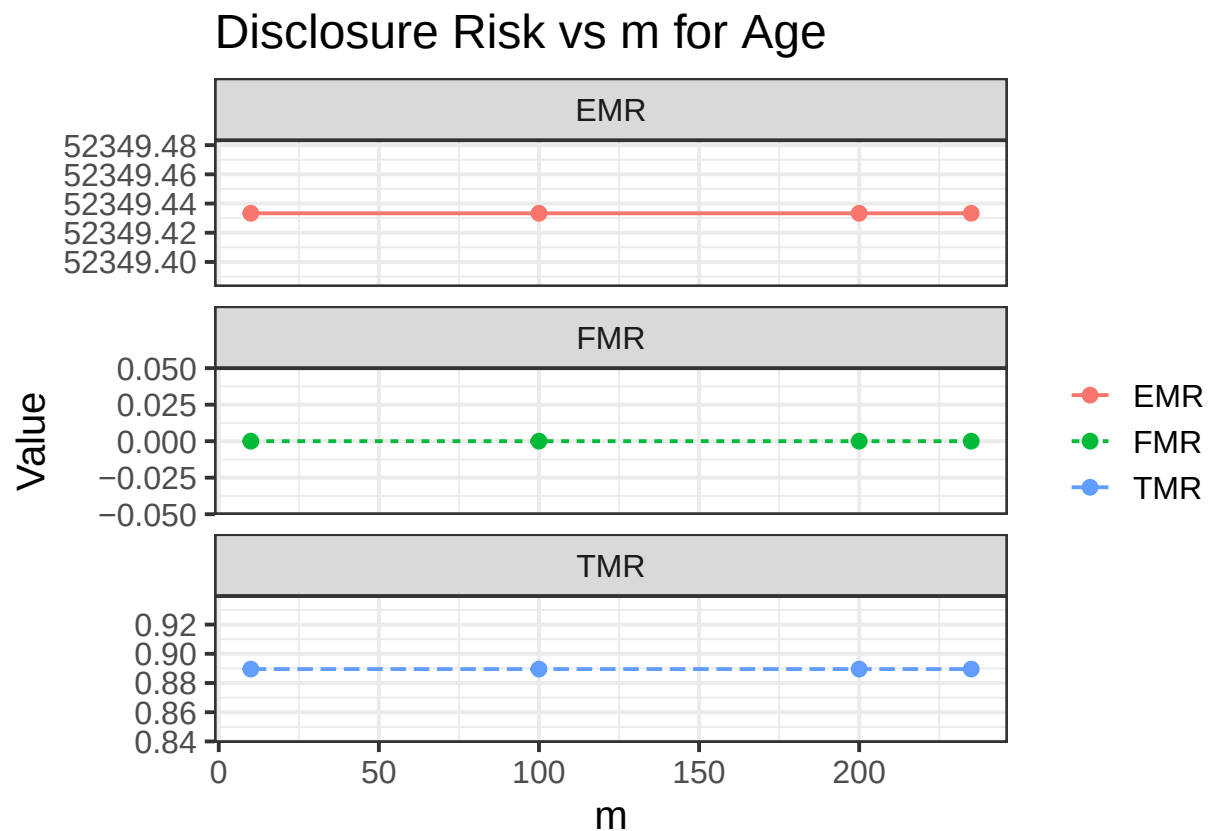
  c(EMR = risk[["EMR"]], TMR = risk[["TMR"]], FMR = risk[["FMR"]])
})
```

```

# Build data frame
risk_df <- tibble(
  m = m_values,
  EMR = risk_values["EMR", ],
  TMR = risk_values["TMR", ],
  FMR = risk_values["FMR", ]
)
risk_df_long <- risk_df %>%
  pivot_longer(cols = c(EMR, TMR, FMR),
    names_to = "measure",
    values_to = "value")

ggplot(risk_df_long, aes(x = m, y = value, colour = measure, linetype = measure)) +
  geom_line() +
  geom_point() +
  facet_wrap(~ measure, scales = "free_y", ncol = 1) +
  xlab("m") +
  ylab("Value") +
  ggtitle("Disclosure Risk vs m for Age") +
  theme_bw(base_size = 15) +
  theme(legend.title = element_blank())

```



Analysis For Age Starting with our initial density plots for Age we can clearly see that $m=10$ is the worst fit, failing to capture both the left tail and the overall shape of the distribution compared to the other m values. The remaining m values are relatively similar in their fit, but from the plots $m=100$ appears to have a slightly better fit overall. $m=200$ and $m=235$ show slightly higher peaks in the middle

of the distribution, which can be attributed to the increased effect of Laplace noise, with more bins each bin contains fewer observations, making the noise proportionally larger relative to the true bin counts. Although $m=100$ visually appears to best represent the distribution, a purely visual approach is insufficient given how similar the plots are. Similarly to our analysis of Billing Amount, we also examine how the pMSE changes across m values. In this case we see that $m=100$ actually has the highest pMSE while $m=200$ has the lowest. This contradicts our visual findings, which highlights why it is important to look at both the visuals and the quantitative measures together. From this we can conclude that while the marginal distribution of synthetic Age looks best at $m=100$, the relationships between Age and the other variables in the data set are best preserved at $m=200$. Since pMSE accounts for these relationships between variables it is a more comprehensive measure of utility than the density plots alone. Turning to disclosure risk, the results for Age are considerably worse than those for Billing Amount. The EMR is extremely high at approximately 52,349 out of 55,500 records, the TMR is approximately 0.89, and the FMR is nearly zero. What is particularly notable is that all three measures remain completely flat across all m values, meaning the choice of m has essentially no effect on disclosure risk for Age. This is a fundamentally different finding from Billing Amount where at least some variation across m values was observed. This will be discussed further in the final analysis section.

Summary and Next Steps

When looking at a truly continuous and high range variable like Billing Amount we can see that changing the m value does have a meaningful influence on the strength of our synthesis. Looking at our results we see that an m value of 100 grants both the best fit from the density plots as well as the best preservation of relationships between variables as measured by pMSE. If you combine this with its generally low disclosure risk compared to the other m values, overall $m=100$ seems like it would be the best m value for Billing Amount. However this is evidently only one data set but these processes are relatively straightforward and repeatable, so moving forward choosing a number of more data sets and running the same synthesis will help us see the validity of these claims more broadly. Moving onto Age we can see that clearly some problems have arisen. In a sense Age is not a true continuous variable but is rather a discrete variable treated as such. This was done so the data set could be kept constant for easier and more direct comparisons, however this causes issues in our synthesis which show up clearly in our results. While our min and max for this synthesis was 0 to 120, a value that was picked blind to the data set, the actual range of Age is 13 to 89 making this variable relatively highly constrained with only 77 possible integer values. This constraint is ultimately the root cause of the poor disclosure risk results we observed for Age. Because there are so few possible values the synthetic ages almost inevitably land close to the real ones regardless of what m value is used, making it very easy for an intruder to correctly identify records. This is exactly why the disclosure risk plots are completely flat across all m values since m simply has no effect when the variable itself is so constrained. For a variable like Age the histogram based differential privacy synthesizer is not well suited to providing meaningful privacy protection, and alternative synthesis methods designed specifically for discrete or bounded integer variables would likely perform considerably better. This is an important takeaway from this analysis which is that the choice of synthesis method should always be informed by the nature of the variable being synthesized and not just the number of bins or privacy budget chosen. Should these processes be repeated with numerous other data sets, we can see the strength of all these claims and patterns, which I believe is a great next step moving forward.